| | |
|---|---|
| Universidad Galileo | Guatemala 6 de noviembre 2026 |
| Facultad: FISICC | Alumno: Luis Cordova |
| Curso: Microprocesadores | Carnet: 23012122 |
| Sección: A | Hora de Laboratorio: 1:00 a 1:50 |
| Auxiliar: Brandon Eduardo Mendoza Espinoza | Día de Laboratorio: Lunes |

# Laboratorio No. 10 - Input Capture y UART

**Resumen:**

Esta práctica reúne dos funciones distintas de los temporizadores del STM32G431 y las combina con el puerto serial asíncrono. Un canal de PWM genera una señal cuadrada de 5 kHz con un ciclo de trabajo del 50 %, que sirve de referencia para un circuito RC, y un canal de Input Capture mide el período y el ancho de pulso de la señal que ese circuito devuelve [4].

La diferencia respecto a las prácticas anteriores está en que ahora el temporizador no solo produce una señal: también la mide. Dos registros de captura del mismo canal guardan el valor del contador, uno en el flanco de subida y otro en el de bajada, de manera que el período y el ancho de pulso quedan medidos por hardware y el programa únicamente los recoge.

El resultado se consulta desde la computadora. El programa configura USART1 a 115200 baudios con interrupción en la recepción, y según el carácter que llegue responde con el período, la frecuencia, el ancho de pulso o el ciclo de trabajo de la señal medida. Ni la captura ni la recepción utilizan sondeo: las dos se atienden por interrupción.

**Objetivos:**

- Generar una señal cuadrada de 5 kHz con un ciclo de trabajo del 50 % utilizando la función PWM de uno de los temporizadores del microcontrolador.
- Configurar otro temporizador en modo de Input Capture para medir el período y el ancho de pulso de una señal externa, asociando el flanco de subida y el de bajada a registros de captura distintos.
- Inicializar el puerto serial USART1 a 115200 baudios, 8 bits de datos, sin paridad y un bit de parada, con interrupciones en la recepción, e implementar una interfaz que reporte las mediciones a través de una terminal serial.

**Teoría:**

La función de Input Capture invierte el uso habitual de un temporizador. En lugar de comparar el contador con un valor para producir una salida, el bloque vigila la entrada del canal y, cuando detecta el flanco configurado, guarda una copia del contador en el registro TIMx_CCRn [1]. La copia la hace el hardware en el instante del flanco, de modo que la medición no arrastra el tiempo que el programa tarde en reaccionar; la rutina de interrupción puede atenderse varios microsegundos después y el valor capturado sigue siendo exacto.

Un mismo pin puede alimentar dos canales de captura a la vez. El registro TIMx_CCMR1 permite dirigir la entrada TI1 tanto al canal 1 como al canal 2, y TIMx_CCER elige de forma independiente qué flanco detecta cada uno [1]. Si al primero se le asigna el flanco de subida y al segundo el de bajada, un solo pin entrega las dos marcas de tiempo que describen un pulso completo.

Falta un detalle para que esas dos marcas sean directamente el período y el ancho de pulso: el contador debe arrancar de cero en cada flanco de subida. Eso se consigue con el modo esclavo de reinicio, configurado en TIMx_SMCR, que conecta la señal del pin al circuito de disparo del contador y lo pone en cero cada vez que llega un flanco de subida [1]. Con el contador sincronizado de esa forma, el valor capturado en el flanco de bajada es el ancho del pulso y el capturado en el siguiente flanco de subida es el período, sin ninguna resta intermedia.

Del lado de la comunicación, el UART transmite cada byte como una trama: una línea en reposo en alto, un bit de arranque que marca el inicio, los bits de datos del menos al más significativo y uno o más bits de parada. No hay una línea de reloj compartida entre los dos extremos, y de ahí viene el nombre de asíncrono: el receptor reconstruye los bits midiendo tiempo a partir del flanco de arranque. Por eso ambos extremos tienen que coincidir previamente en la velocidad, el número de bits de datos, la paridad y los bits de parada; si no coinciden, lo que llega es basura y no un error declarado.

La velocidad se programa escribiendo en el registro USARTx_BRR el número de pulsos del reloj del periférico que dura un bit [1]. Como ese número casi nunca resulta entero, siempre queda un error de redondeo, y el receptor lo tolera mientras el desfase acumulado a lo largo de la trama no llegue a medio bit.

Para la recepción, el periférico levanta la bandera RXNE cada vez que termina de recibir un byte. Consultarla en un lazo obliga al procesador a esperar sin hacer nada; habilitando el bit RXNEIE la llegada de un dato genera una interrupción y el programa se entera solo cuando hay algo que atender [1]. El enunciado de la práctica pide expresamente esta segunda forma.

El circuito RC, por último, es lo que convierte la señal de referencia en algo que vale la pena medir. El capacitor se carga y se descarga siguiendo la señal cuadrada, pero lo hace de forma exponencial, no instantánea. El pin del microcontrolador no ve esa rampa como tal: su entrada es un disparador de Schmitt que conmuta al cruzar un umbral, de modo que la rampa vuelve a convertirse en una señal digital, pero con sus flancos desplazados en el tiempo. Ese desplazamiento depende de la constante de tiempo del circuito, y es la variable que el Input Capture termina midiendo.

**Resultados prácticos:**

A continuación se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

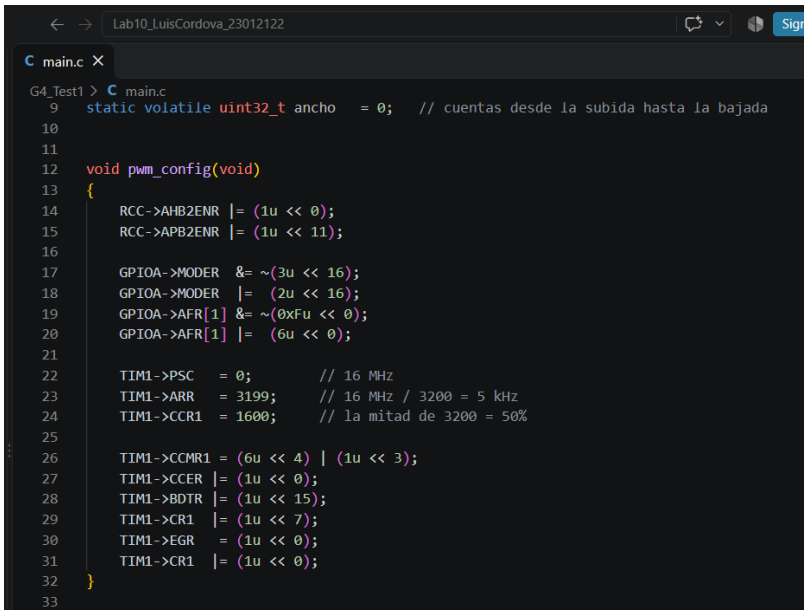



Ilustración 1- subrutina pwm_config: PA8 en función alterna y TIM1 programado para 5 kHz con un ciclo de trabajo del 50 %.

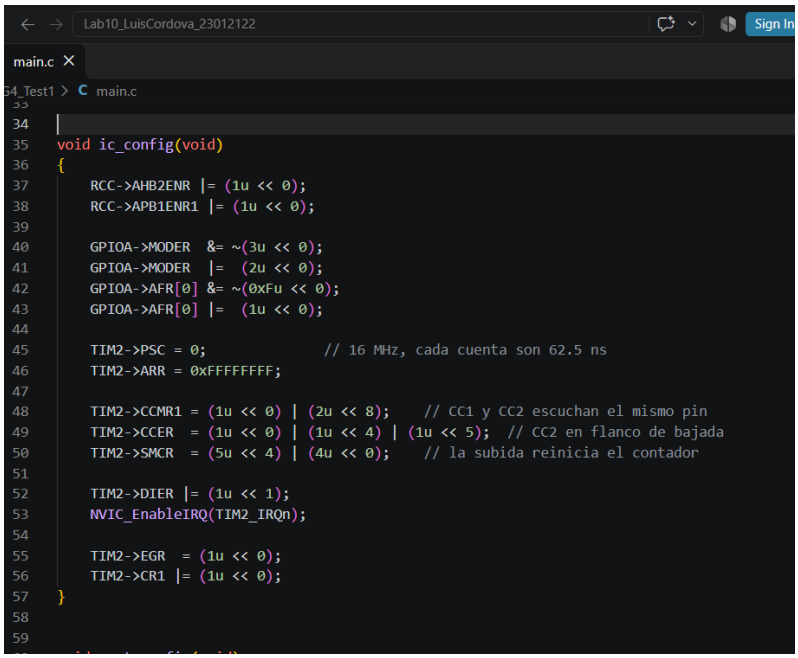



Ilustración 2- subrutina ic_config: PA0 en función alterna, los dos canales de captura sobre la misma entrada y el modo de reinicio del contador.

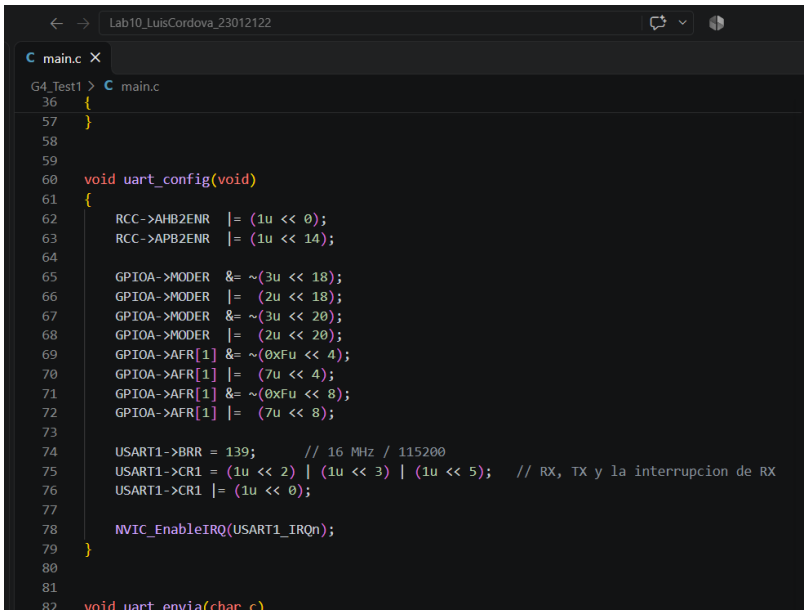



Ilustración 3- subrutina uart_config: PA9 y PA10 en función alterna, USART1 a 115200 baudios y la interrupción de recepción habilitada.




Ilustración 4- subrutinas uart_envia y uart_texto, que escriben un carácter y una cadena en el registro de transmisión.

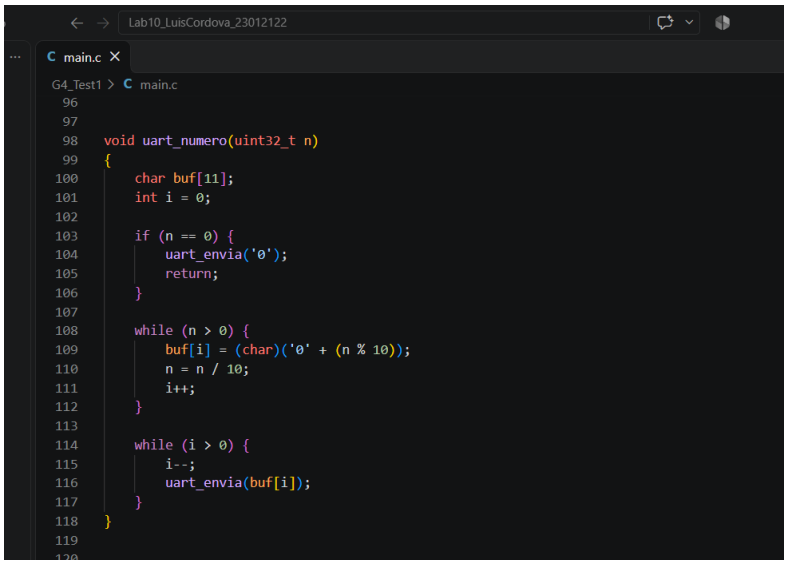



Ilustración 5- subrutina uart_numero, que convierte un entero sin signo en su representación decimal para enviarlo por el puerto serial.

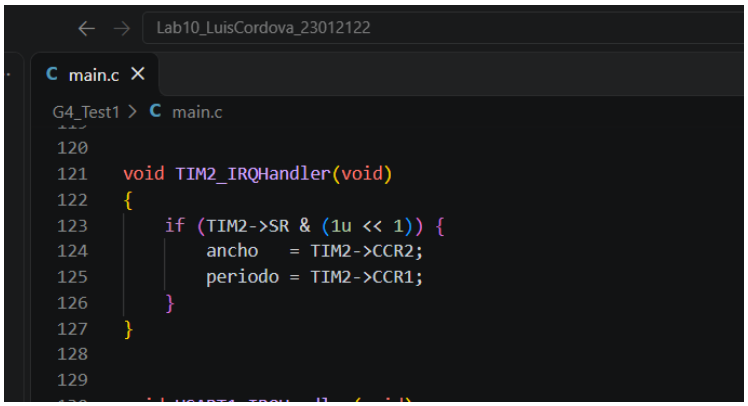



Ilustración 6- rutina de servicio de la interrupción del temporizador, donde se recogen las dos capturas.

```
Lab10_LuisCordova_23012122
main.c
G4_Test1 > main.c
128
129
130  void USART1_IRQHandler(void)
131  {
132      char c;
133
134      if (USART1->ISR & (1u << 5)) {
135          c = (char)USART1->RDR;
136
137          if (periodo == 0) {
138              uart_texto("No hay senal\r\n");
139              return;
140          }
141
142          if (c == '0') {
143              uart_texto("Periodo = ");
144              uart_numero(periodo / 16);
145              uart_texto(" us\r\n");
146          }
147          else if (c == '1') {
148              uart_texto("Frecuencia = ");
149              uart_numero(16000000u / periodo);
150              uart_texto(" Hz\r\n");
151          }
152          else if (c == '2') {
153              uart_texto("Ancho de pulso = ");
154              uart_numero(ancho / 16);
155              uart_texto(" us\r\n");
156          }
157          else if (c == '3') {
158              uart_texto("Duty Cycle = ");
159              uart_numero((ancho * 100u) / periodo);
160              uart_texto("%\r\n");
161          }
162      }
163  }
```

Ilustración 7- rutina de servicio de la interrupción del puerto serial, con la respuesta correspondiente a cada uno de los cuatro caracteres.

```
Lab10_LuisCordova_23012122
main.c
G4_Test1 > main.c
164
165
166  int main(void)
167  {
168      pwm_config();
169      ic_config();
170      uart_config();
171
172      while (1) {
173          __WFI();
174      }
175  }
176
```

Ilustración 8- función main con las tres configuraciones y el lazo principal.

## Resultados Teóricos:

## Diseño del circuito

El ejemplo del enunciado corresponde a la tarjeta de evaluación del STM32F303 y mide un sensor de humedad conectado a PB0, con la referencia saliendo por PC8 [4]. La Nucleo-G431KB solo expone pines de los puertos A y B, más PF0 y PF1, así que la asignación tuvo que rehacerse [2]. Los pines del puerto serial sí vienen impuestos por el enunciado, que pide expresamente PA9 como transmisión y PA10 como recepción.

La asignación final quedó así:

| Elemento | Pin | Rótulo | Función |
|---|---|---|---|
| Señal de referencia | PA8 | D9 | TIM1_CH1 |
| Entrada de medición | PA0 | A0 | TIM2_CH1 |
| Transmisión | PA9 | D1 | USART1_TX |
| Recepción | PA10 | D0 | USART1_RX |

El divisor se armó con una resistencia de 82.5 kΩ entre PA8 y el nodo de medición, y un capacitor entre ese nodo y tierra. El nodo es el que se conecta a PA0. El enunciado pide capacitores de entre 100 pF y 220 pF, y el orden de magnitud no es negociable: con 82.5 kΩ y 220 pF la constante de tiempo vale

$$\tau = R \cdot C = 82\,500\ \Omega \times 220 \times 10^{-12}\ \text{F} \approx 18.2\ \mu\text{s}$$

frente a un semiperíodo de 100 µs, de modo que el capacitor alcanza a cargarse y descargarse por completo dentro de cada medio ciclo. Con un capacitor de 100 nF, que es el valor que uno tiene a mano con más frecuencia, la constante de tiempo subiría a 8.25 ms, el capacitor nunca llegaría al umbral del pin y no habría ningún flanco que capturar.

**Elección de los temporizadores y de los pines**

Ninguno de los dos pines podía escogerse libremente: cada uno tiene que estar asociado por la tabla de funciones alternas al canal del temporizador que se quiere usar. PA8 está conectado a TIM1_CH1 por la función alterna 6, y PA0 a TIM2_CH1 por la función alterna 1 [8].

Para la señal de referencia se reutilizó TIM1, el mismo de la práctica anterior, con la consideración de siempre en los temporizadores de control avanzado: la salida permanece inhibida hasta que se habilita el bit MOE del registro TIM1_BDTR [1]. Para la medición se escogió TIM2, que es el único temporizador de 32 bits disponible en este microcontrolador [8]; con un contador tan amplio no hay riesgo de desbordamiento aunque la señal medida se detenga o baje mucho de frecuencia.

**Frecuencia de la señal de referencia**

El período de la señal lo fijan en conjunto el preescalador y el registro de autorrecarga, mediante la relación

$$\text{ARR} = f_{clk} / \left( (\text{PSC} + 1) \times f_{PWM} \right) - 1$$

Con el reloj del sistema en 16 MHz, sin preescalador y para los 5 kHz que pide el enunciado:

$$ARR = 16\,000\,000 / ( 1 \times 5\,000 ) - 1 = 3\,199$$

El ciclo de trabajo del 50 % se obtiene escribiendo en TIM1_CCR1 la mitad del período completo, es decir 1 600 de las 3 200 cuentas que dura un ciclo. Dejar el preescalador en cero no es un detalle menor: hace que cada cuenta del temporizador valga lo mismo que cada cuenta del que mide, lo que simplifica la comparación entre la señal generada y la medida.

**Medición con dos registros de captura**

La configuración de TIM2 se reparte entre tres registros, y cada uno aporta una pieza del mecanismo:

| Registro | Campo | Valor | Efecto |
|---|---|---|---|
| TIM2_CCMR1 | CC1S | 01 | el canal 1 escucha la entrada TI1 |
| TIM2_CCMR1 | CC2S | 10 | el canal 2 escucha también TI1 |
| TIM2_CCER | CC1P | 0 | el canal 1 captura en el flanco de subida |
| TIM2_CCER | CC2P | 1 | el canal 2 captura en el flanco de bajada |
| TIM2_SMCR | TS | 101 | la fuente de disparo es la entrada TI1 |
| TIM2_SMCR | SMS | 100 | modo de reinicio: el disparo pone el contador en cero |

El resultado de esa combinación es que el contador vuelve a cero en cada flanco de subida de la señal medida. TIM2_CCR2 queda entonces con la cantidad de cuentas transcurridas hasta el flanco de bajada, que es el ancho del pulso, y TIM2_CCR1 con las transcurridas hasta el siguiente flanco de subida, que es el período completo.

Dentro de la rutina de servicio los dos registros se leen en un orden determinado: primero TIM2_CCR2 y después TIM2_CCR1. La razón es que leer el registro de captura del canal 1 borra por hardware su bandera, de modo que si se leyera primero se perdería la referencia del evento antes de haber recogido el otro dato.

Las dos variables donde se guardan las capturas se declararon volatile, porque la rutina de interrupción las escribe y la rutina del puerto serial las lee. Sin ese calificativo el compilador podría suponer que su contenido no cambia entre dos lecturas y conservar el valor en un registro.

**Resolución del contador y conversión a unidades**

Con el preescalador de TIM2 en cero, el contador avanza al ritmo del reloj del sistema, de manera que cada cuenta representa

$$1 / 16\,000\,000 \text{ Hz} = 62.5 \text{ ns}$$

Los valores capturados están en cuentas y hay que llevarlos a unidades de tiempo. Como hay exactamente dieciséis cuentas por microsegundo, la conversión se reduce a una división entera, sin punto flotante ni constantes intermedias:

$$\text{tiempo } [\mu s] = \text{cuentas} / 16$$

La frecuencia y el ciclo de trabajo se calculan directamente sobre las cuentas, sin pasar por el tiempo, para no acumular el error del redondeo:

$$f = 16\,000\,000 / \text{periodo} \qquad D = \text{ancho} \times 100 / \text{periodo}$$

Para la señal esperada, de 200 µs de período, eso significa 3 200 cuentas, un número lo bastante grande como para que el redondeo a enteros no se note: la resolución de 62.5 ns equivale a tres centésimas de punto porcentual en el ciclo de trabajo.

**Efecto del circuito RC sobre el ancho de pulso**

El capacitor se carga siguiendo la curva exponencial habitual, y el pin conmuta cuando el voltaje del nodo cruza su umbral. Si se llama $V_{IH}$ al umbral de subida y $V_{IL}$ al de bajada, y se admite que el capacitor alcanza prácticamente los extremos dentro de cada semiperíodo, el flanco de subida que ve el pin se atrasa

$$t_s = \tau \cdot \ln( V_{DD} / (V_{DD} - V_{IH}) )$$

y el de bajada se atrasa

$$t_b = \tau \cdot \ln( V_{DD} / V_{IL} )$$

El período medido no cambia, porque los dos flancos de subida consecutivos se atrasan lo mismo. El ancho de pulso sí, y resulta

$$\text{ancho} = T/2 + \tau \cdot \ln( (V_{DD} - V_{IH}) / V_{IL} )$$

La expresión deja ver el fundamento de la medición. Si los dos umbrales estuvieran colocados simétricamente respecto de la alimentación, el logaritmo se anularía y el ciclo de trabajo seguiría siendo del 50 % sin importar el valor del capacitor. Son la asimetría de los umbrales de la entrada y la constante de tiempo las que, juntas, convierten un cambio de capacitancia en un cambio del ancho de pulso medido, y por eso la figura del enunciado muestra la señal del sensor con la misma frecuencia que la de referencia pero con un ciclo de trabajo mayor [4].

De ahí sale también el criterio para dimensionar el circuito. El término que contiene la información es proporcional a $\tau$, de modo que un capacitor demasiado pequeño deja el ancho de pulso indistinguible de la mitad del período, y uno demasiado grande impide que el capacitor llegue al umbral. El rango que propone el enunciado, entre 100 pF y 220 pF con 82.5 kΩ, mantiene el circuito dentro de esa ventana.

**Configuración del puerto serial**

El enunciado fija la configuración en 115200 baudios, 8 bits de datos, sin paridad y un bit de parada [4]. Los tres últimos parámetros son los valores que el periférico toma al reiniciarse, así que no requieren ninguna escritura: basta con no modificar los bits M0 y M1 del registro USART1_CR1, el bit de control de paridad y el campo STOP de USART1_CR2.

La velocidad sí hay que programarla. Con el reloj del periférico en 16 MHz:

$$\text{BRR} = 16\,000\,000 / 115\,200 = 138.9 \rightarrow 139$$

El redondeo a 139 deja una velocidad real de 115 108 baudios, un error del 0.08 % respecto de la nominal. Sobre una trama de diez bits el desfase acumulado es inferior a la centésima parte de un bit, muy por debajo del medio bit que el receptor puede tolerar, de modo que el redondeo no compromete la comunicación.

De USART1_CR1 se habilitan tres bits antes de encender el periférico: RE para la recepción, TE para la transmisión y RXNEIE para que la llegada de un byte genere una interrupción. El orden importa, porque algunos campos de configuración solo admiten escritura mientras el periférico está deshabilitado [1].

**La interfaz de comandos**

La rutina de servicio del puerto serial lee el carácter recibido y responde según la tabla que pide el enunciado:

| Carácter | Respuesta | Cálculo |
|---|---|---|
| 0 | Periodo = 200 us | periodo / 16 |
| 1 | Frecuencia = 5000 Hz | 16 000 000 / periodo |
| 2 | Ancho de pulso = ... us | ancho / 16 |
| 3 | Duty Cycle = ... % | ancho × 100 / periodo |

Leer el registro USART1_RDR borra la bandera de recepción por hardware, de modo que la rutina no necesita limpiarla de forma explícita. Antes de calcular nada se comprueba que la variable del período no valga cero, que es el estado en que queda si todavía no se ha capturado ningún flanco; sin esa comprobación las dos divisiones de la tabla producirían una división entre cero.

El envío se resolvió con tres subrutinas pequeñas. La primera espera a que el registro de transmisión quede libre y escribe un carácter, la segunda recorre una cadena apoyándose en la primera, y la tercera convierte un entero sin signo a decimal extrayendo sus dígitos por divisiones sucesivas entre diez. Como los dígitos salen del menos significativo al más significativo, se guardan en un arreglo y se envían en orden inverso.

La espera dentro de la subrutina de envío es el único lazo de sondeo del programa, y no contradice la indicación del enunciado: lo que se pide evitar es sondear la recepción, que es lo que mantendría

al procesador esperando un evento externo de duración desconocida. El registro de transmisión, en cambio, se libera en un tiempo acotado y conocido.

**Estructura del programa**

Con las dos interrupciones configuradas, el lazo principal queda sin trabajo. La función main se limita a llamar las tres configuraciones y a entrar en un ciclo que ejecuta la instrucción WFI, la cual detiene al procesador hasta que llega una interrupción. El programa reacciona solo a dos eventos: un flanco en el pin de medición y la llegada de un carácter por el puerto serial.

**Modificación del paso 9**

El procedimiento pide borrar todas las líneas que estén dentro de la rutina de servicio del temporizador y volver a cargar el programa. Con esa modificación se pierden precisamente las dos asignaciones que recogen las capturas.

Lo interesante es cuánto sigue funcionando. La señal de referencia continúa saliendo por PA8, porque la produce TIM1 sin intervención del programa. El bloque de captura de TIM2 sigue detectando los flancos y copiando el contador en sus registros, también por hardware. El puerto serial sigue respondiendo a las teclas. Lo único que deja de ocurrir es el traslado de los registros de captura a las variables, de modo que el período y el ancho se quedan congelados en el último valor recogido antes de la modificación; si el programa se carga de nuevo desde el arranque, las variables conservan su valor inicial de cero y la aplicación responde que no hay señal.

Hay además un segundo efecto, menos evidente. La bandera de captura del canal 1 se borra al leer el registro correspondiente, y esa lectura era justamente una de las líneas eliminadas. Sin ella la bandera queda permanentemente levantada y la interrupción se vuelve a solicitar de inmediato en cuanto se sale de la rutina, con lo que el procesador queda atrapado atendiéndola una y otra vez. El puerto serial, que tiene la misma prioridad por omisión, deja de ser atendido. La demostración resulta más contundente de lo que parece: no solo se congela la medición, se detiene la aplicación entera.

**Conclusiones:**

- El Input Capture traslada la medición de tiempo al hardware. El contador se copia en el instante exacto del flanco, de modo que la precisión del resultado no depende de lo que el programa tarde en atender la interrupción, que es justamente la limitación de medir con lazos de software.
- Dirigir dos canales de captura a la misma entrada y reiniciar el contador en cada flanco de subida permite obtener el período y el ancho de pulso directamente de dos registros, sin ninguna resta ni ningún cálculo intermedio. Es un buen ejemplo de cómo una configuración adecuada del periférico reduce el programa a casi nada.
- Escoger el preescalador de ambos temporizadores en cero, y el registro de autorrecarga de manera que cada microsegundo corresponda a dieciséis cuentas, convirtió todas las

conversiones en divisiones enteras exactas. Elegir las escalas antes de escribir el programa evita arrastrar factores de corrección y redondeos en cada operación.

**Bibliografía:**


[1] STMicroelectronics, «STM32G4 series advanced Arm-based 32-bit MCUs», Reference manual RM0440, Rev. 9, marzo de 2025, capítulos 27, 28 y 37. [En línea]. Disponible en: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-arm-based-32bit-mcus-stmicroelectronics.pdf

[2] STMicroelectronics, «STM32G4 Nucleo-32 board (MB1430)», User manual UM2397, Rev. 2, septiembre de 2019. [En línea]. Disponible en: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf

[3] STMicroelectronics, «STM32 Cortex-M4 MCUs and MPUs programming manual», Programming manual PM0214. [En línea]. Disponible en: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf

[4] J. Fajardo, «Práctica de Laboratorio #10: Input Capture y UART», Microprocesadores, Universidad Galileo, noviembre de 2026.

[5] STMicroelectronics, «STM32G431x6/x8/xB datasheet», DS12589, características de las entradas y salidas. [En línea]. Disponible en: https://www.st.com/resource/en/datasheet/stm32g431kb.pdf

[6] STMicroelectronics, «stm32g4xx_hal_driver», funciones HAL_TIM_IC_Init y HAL_UART_Init. [En línea]. Disponible en: https://github.com/STMicroelectronics/stm32g4xx_hal_driver

[7] Arm Limited, «Arm Compiler for Embedded User Guide», Arm Compiler 6. [En línea]. Disponible en: https://developer.arm.com/documentation/

[8] STMicroelectronics, «cmsis_device_g4», archivo stm32g431xx.h, definiciones de periféricos y funciones alternas del dispositivo. [En línea]. Disponible en: https://github.com/STMicroelectronics/cmsis_device_g4